DFS and BFS: Detailed Step-by-Step Guide Exercises 1.2, 1.3, 1.4

Exercise 3.1 - Detailed Solutions

1 Introduction

This document provides **in-depth, step-by-step execution** of Depth-First Search (DFS) and Breadth-First Search (BFS) algorithms for the problem formulations from Exercises 1.2, 1.3, and 1.4.

Coverage:

- **Exercise 1.2:** Team Assignment Problem
- **Exercise 1.3:** Meeting Point Problem
- **Exercise 1.4:** Elevator Scheduling Problem

For each problem, we will:

1. Present the problem formulation
2. Show detailed DFS execution (5-10 steps)
3. Show detailed BFS execution (5-10 steps)
4. Compare the behavior of both algorithms

2 Gate Smashers Explanation

BFS - The Gate Smashers Way

Before diving into the formal algorithms, let's understand BFS the intuitive way!

What is Uninformed Search?

BFS is an **uninformed search technique** (also called **blind search** or **brute force method**). **Uninformed** means: *There is no domain-specific knowledge.*

- We know how to go from A to B (the cost from A to B)
- But we **don't know** if B will take us to the goal state
- We don't know in advance how much it will cost to reach the goal from B
- We just know that *someday* we will find the goal!

This is different from **informed/heuristic search** where we have an estimate of the cost to reach the goal from any state.

FIFO and Queue Data Structure

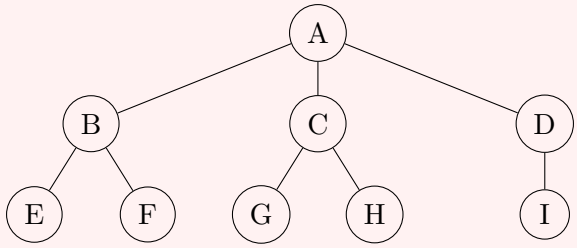
BFS uses a **Queue** which works on **FIFO (First In First Out)**:

- We **insert elements from the tail** (back of queue)

- We **remove elements from the head** (front of queue)
- First element that goes in is the first element that comes out!

How BFS Works - Simple Example

Consider this tree:



Step-by-step Queue Operations:

1. Start: Queue = $[A]$
2. Remove A, add its children: Queue = $[B, C, D]$
3. Remove B, add its children: Queue = $[C, D, E, F]$
4. Remove C, add its children: Queue = $[D, E, F, G, H]$
5. Remove D, add its children: Queue = $[E, F, G, H, I]$
6. Continue removing E, F, G, H, I...

Order of visiting: A → B → C → D → E → F → G → H → I

Key Insight: Level-by-Level (Shallowest Node First)

This is the most important point!
BFS explores **level by level** (also called **level order traversal**):

- First, complete Level 0 (root)
- Then, complete Level 1 (all children of root)
- Then, complete Level 2 (all grandchildren)
- And so on...

Shallowest node means the **nearest node** - BFS always explores the closest nodes before going deeper!
Comparison with DFS:

- **DFS** goes deep in one direction first (might get stuck in infinite depth)
- **BFS** completes each level before moving to the next (explores breadth first)

Important Properties

1. **Complete:** BFS will *definitely* find the goal if it exists!
   - We don't leave any element behind
   - We complete all levels one by one
   - Even if search space is large, we will find it
2. **Optimal:** BFS gives the *shortest path*!
   - When all edge costs are the same (e.g., all = 1)
   - It always gives the shortest/optimal result
3. **Time Complexity:** $O(b^d)$
   - b = **branching factor** (maximum number of children any node has)
   - d = **depth** (how many levels from root to goal)
   - Example: If $b = 3$ and $d = 2$, then $3^2 = 9$ nodes maximum to search

Finding the Path

When we find the goal element:

1. Check: Is this the element we are searching for? (Goal test)
2. If yes, take its **parent node pointer**
3. Trace back: Goal → Parent → Grandparent → ... → Root
4. This gives us the path!

3 Algorithm Review

3.1 Depth-First Search (DFS)

DFS Tree Search

DFS-TreeSearchΘ $n_0 \leftarrow$ InitNode(InitialState(Θ)) $open \leftarrow [n_0]$ STACK (LIFO) $open \neq \emptyset$ $n \leftarrow open$.Pop() Remove LAST element GoalTest($n.State, \Theta$) ExtractSolution(n) $a \in$ Actions($n.State, \Theta$) $s' \leftarrow$ Successor($n.State, a, \Theta$) $n' \leftarrow$ SuccNode(n, a, s', Θ) $open$.Push(n') Add to END (top) **failure**

3.2 Breadth-First Search (BFS)

BFS Tree Search

BFS-TreeSearchΘ $n_0 \leftarrow$ InitNode(InitialState(Θ)) $open \leftarrow [n_0]$ QUEUE (FIFO) $open \neq \emptyset$ $n \leftarrow open$.Dequeue() Remove FIRST element GoalTest($n.State, \Theta$) ExtractSolution(n) $a \in$ Actions($n.State, \Theta$) $s' \leftarrow$ Successor($n.State, a, \Theta$) $n' \leftarrow$ SuccNode(n, a, s', Θ) $open$.Enqueue(n') Add to END (back) **failure**

4 Exercise 1.2: Team Assignment Problem

4.1 Problem Formulation

Team Assignment Problem

Problem: We have 6 engineers (A-F) with skill levels in Front-end, Back-end, and DevOps. We want to form two teams of exactly 3 members each so that skill levels are balanced.
Balance Constraint: For each skill, the difference between team sums cannot exceed 2.

Engineer	Front-end	Back-end	DevOps
A	5	2	1
B	3	4	2
C	2	5	3
D	4	3	2
E	1	4	5
F	3	2	4

State Representation: Binary vector (A, B, C, D, E, F) where 0 = Team 1, 1 = Team 2
Example State: $(0, 1, 1, 1, 0, 0)$ means:

- Team 1: A, E, F
- Team 2: B, C, D

Initial State: Any state, e.g., $(0, 0, 0, 0, 0, 0)$ (all in Team 1)
Actions: For each engineer X: `X-to-0` and `X-to-1`, cost = 1
Goal Test: Check if:

1. Each team has exactly 3 members
2. For each skill: $|\text{Team1_sum} - \text{Team2_sum}| \leq 2$

4.2 DFS Execution

DFS on Team Assignment - Step by Step

Initial State: $(A = 0, B = 0, C = 0, D = 0, E = 0, F = 0)$ - All in Team 1
Action Ordering: Alphabetically: A-to-1, B-to-1, C-to-1, D-to-1, E-to-1, F-to-1

Step 0: Initialize

Open (Stack)	$n_0 : (0, 0, 0, 0, 0, 0), g = 0$
Expanded	—

Step 1: Expand n_0

Pop: $n_0 : (0, 0, 0, 0, 0, 0)$
Goal Test: NO - only Team 1 has members (6 members), not balanced
Available Actions: A-to-1, B-to-1, C-to-1, D-to-1, E-to-1, F-to-1
Generate Successors:

- `A-to-1`: $(1, 0, 0, 0, 0, 0) \rightarrow n_1, g = 1$
- `B-to-1`: $(0, 1, 0, 0, 0, 0) \rightarrow n_2, g = 1$
- `C-to-1`: $(0, 0, 1, 0, 0, 0) \rightarrow n_3, g = 1$

- `D-to-1`: $(0,0,0,1,0,0) \to n_4, g = 1$
- `E-to-1`: $(0,0,0,0,1,0) \to n_5, g = 1$
- `F-to-1`: $(0,0,0,0,0,1) \to n_6, g = 1$

Open (Stack)	$[n_1, n_2, n_3, n_4, n_5, \mathbf{n_6}] \leftarrow$ top
Expanded	n_0

DFS: n_6 was added last, so it's popped next!

Step 2: Expand n_6

Pop: $n_6 : (0,0,0,0,0,1)$ - F moved to Team 2
Goal Test: NO - Team 2 has 1 member, Team 1 has 5
Available Actions: A-to-1, B-to-1, C-to-1, D-to-1, E-to-1, F-to-0
Generate Successors:

- `A-to-1`: $(1,0,0,0,0,1) \to n_7, g = 2$
- `B-to-1`: $(0,1,0,0,0,1) \to n_8, g = 2$
- `C-to-1`: $(0,0,1,0,0,1) \to n_9, g = 2$
- `D-to-1`: $(0,0,0,1,0,1) \to n_{10}, g = 2$
- `E-to-1`: $(0,0,0,0,1,1) \to n_{11}, g = 2$
- `F-to-0`: $(0,0,0,0,0,0) \to$ already seen (skip in tree search)

Open (Stack)	$[n_1, n_2, n_3, n_4, n_5, n_7, n_8, n_9, n_{10}, \mathbf{n_{11}}] \leftarrow$ top
Expanded	n_0, n_6

Going Deep: DFS continues down the F=1 branch first!

Step 3: Expand n_{11}

Pop: $n_{11} : (0,0,0,0,1,1)$ - E and F in Team 2
Goal Test: NO - Team 2 has 2 members, Team 1 has 4
Available Actions: A-to-1, B-to-1, C-to-1, D-to-1, E-to-0, F-to-0
Generate Successors:

- `A-to-1`: $(1,0,0,0,1,1) \to n_{12}, g = 3$ - 3 in each team!
- `B-to-1`: $(0,1,0,0,1,1) \to n_{13}, g = 3$ - 3 in each team!
- `C-to-1`: $(0,0,1,0,1,1) \to n_{14}, g = 3$ - 3 in each team!
- `D-to-1`: $(0,0,0,1,1,1) \to n_{15}, g = 3$ - 3 in each team!
- (other actions lead back)

Open (Stack)	$[n_1, ..., n_{10}, n_{12}, n_{13}, n_{14}, \mathbf{n_{15}}] \leftarrow$ top
Expanded	n_0, n_6, n_{11}

Progress: Generated states with 3 members per team!

Step 4: Expand n_{15} - Check Goal

Pop: $n_{15} : (0, 0, 0, 1, 1, 1)$
Teams:

- Team 1: A, B, C
- Team 2: D, E, F

Skill Sums:

- Team 1 Front-end: 5+3+2 = 10
- Team 2 Front-end: 4+1+3 = 8 $\rightarrow$ Diff = 2 ✓
- Team 1 Back-end: 2+4+5 = 11
- Team 2 Back-end: 3+4+2 = 9 $\rightarrow$ Diff = 2 ✓
- Team 1 DevOps: 1+2+3 = 6
- Team 2 DevOps: 2+5+4 = 11 $\rightarrow$ Diff = 5 ×

Goal Test: **FAIL** - DevOps difference is 5 ¿ 2
Continue expanding...

Step 5: Expand n_{14}

Pop: $n_{14} : (0, 0, 1, 0, 1, 1)$
Teams:

- Team 1: A, B, D
- Team 2: C, E, F

Skill Sums:

- Team 1 Front-end: 5+3+4 = 12
- Team 2 Front-end: 2+1+3 = 6 $\rightarrow$ Diff = 6 ×

Goal Test: **FAIL** - Front-end difference is 6 ¿ 2
Continue expanding...

Step 6: Expand n_{13}

Pop: $n_{13} : (0, 1, 0, 0, 1, 1)$
Teams:

- Team 1: A, C, D
- Team 2: B, E, F

Skill Sums:

- Team 1 Front-end: 5+2+4 = 11
- Team 2 Front-end: 3+1+3 = 7 $\rightarrow$ Diff = 4 ×

Goal Test: **FAIL**
Continue expanding...

Step 7: Expand n_{12} - POTENTIAL GOAL!

Pop: $n_{12} : (1, 0, 0, 0, 1, 1)$
Teams:

- Team 1: B, C, D
- Team 2: A, E, F

Skill Sums:

- Team 1 Front-end: 3+2+4 = 9
- Team 2 Front-end: 5+1+3 = 9 $\rightarrow$ Diff = 0 ✓
- Team 1 Back-end: 4+5+3 = 12
- Team 2 Back-end: 2+4+2 = 8 $\rightarrow$ Diff = 4 ×

Goal Test: **FAIL** - Back-end difference is 4 ¿ 2

Note: DFS would continue exploring. In a real execution, it might eventually find a balanced solution or determine none exists along this path.

4.3 BFS Execution

BFS on Team Assignment - Step by Step

Initial State: $(A = 0, B = 0, C = 0, D = 0, E = 0, F = 0)$ - All in Team 1

Step 0: Initialize

Open (Queue)	$[\mathbf{n_0} : (0, 0, 0, 0, 0, 0), g = 0]$
Expanded	—
Level	0

Step 1: Expand n_0 - Level 0

Dequeue: $n_0 : (0, 0, 0, 0, 0, 0)$
Goal Test: NO
Generate Successors: All 6 actions (move each engineer to Team 2)

- $n_1 : (1, 0, 0, 0, 0, 0), g = 1$ - A to Team 2
- $n_2 : (0, 1, 0, 0, 0, 0), g = 1$ - B to Team 2
- $n_3 : (0, 0, 1, 0, 0, 0), g = 1$ - C to Team 2
- $n_4 : (0, 0, 0, 1, 0, 0), g = 1$ - D to Team 2
- $n_5 : (0, 0, 0, 0, 1, 0), g = 1$ - E to Team 2
- $n_6 : (0, 0, 0, 0, 0, 1), g = 1$ - F to Team 2

Open (Queue)	$[\mathbf{n_1}, n_2, n_3, n_4, n_5, n_6]$
Expanded	n_0
Level	1 - All depth-1 nodes in queue

BFS: All children of root now in queue, will explore level-by-level!

Step 2: Expand n_1 - Level 1

Dequeue: $n_1 : (1, 0, 0, 0, 0, 0)$ - A in Team 2
Goal Test: NO
Generate Successors: Move another engineer to Team 2 or move A back

- `B-to-1`: $n_7 : (1, 1, 0, 0, 0, 0), g = 2$
- `C-to-1`: $n_8 : (1, 0, 1, 0, 0, 0), g = 2$
- `D-to-1`: $n_9 : (1, 0, 0, 1, 0, 0), g = 2$
- `E-to-1`: $n_{10} : (1, 0, 0, 0, 1, 0), g = 2$
- `F-to-1`: $n_{11} : (1, 0, 0, 0, 0, 1), g = 2$
- (A-to-0 goes back to initial state)

Open (Queue)	$[\mathbf{n_2}, n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}, n_{11}]$
Expanded	n_0, n_1
Level	Still expanding level 1; level 2 nodes added to back

Step 3-6: Continue Level 1

BFS continues expanding all level-1 nodes: n_2, n_3, n_4, n_5, n_6
Each generates 5 new level-2 nodes (moving a second engineer to Team 2).
After expanding all level-1 nodes:

- All single-engineer-in-Team-2 states explored
- Queue now contains many level-2 nodes (two engineers in Team 2)

Level 1 Complete! Now BFS explores level 2.

Step 7: Expand Level 2 - Two Engineers in Team 2

Example - Dequeue: $n_7 : (1, 1, 0, 0, 0, 0)$ - A, B in Team 2
Goal Test: NO - Team sizes are 4 vs 2
Generate Successors: Move a third engineer to Team 2
This creates level-3 nodes with 3 engineers per team - potential goals!

- $n_x : (1, 1, 1, 0, 0, 0), g = 3$ - A, B, C in Team 2
- $n_y : (1, 1, 0, 1, 0, 0), g = 3$ - A, B, D in Team 2
- $n_z : (1, 1, 0, 0, 1, 0), g = 3$ - A, B, E in Team 2
- etc.

Progress: BFS systematically explores all 2-engineer combinations before checking any 3-engineer combinations!

BFS Key Observation

BFS Behavior:

- Level 0: Initial state (all in Team 1)

- Level 1: 6 states (one engineer in Team 2)
- Level 2: $\binom{6}{2} = 15$ states (two engineers in Team 2)
- Level 3: $\binom{6}{3} = 20$ states (three engineers in Team 2) - CHECK ALL!

Guaranteed to find solution at level 3 if one exists, because:

1. All level-3 states have 3 members per team
2. BFS explores ALL such states systematically
3. Will find the first (shallowest) balanced configuration

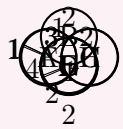
Comparison with DFS: DFS dove deep into one branch (F, then E+F, etc.), while BFS explored breadth-first, ensuring optimal depth.

5 Exercise 1.3: Meeting Point Problem

5.1 Problem Formulation

Meeting Point Problem

Problem: Alice, Emil, and Frank want to meet somewhere in the city. Find the meeting point and route for each friend to minimize total distance traveled.



State Representation: (a, e, f) where:

- a = Alice's current location
- e = Emil's current location
- f = Frank's current location

Initial State: (A, E, F) - Alice at A, Emil at E, Frank at F
Goal States: (X, X, X) for any location X - all at same location
Actions:

- `move-a-X-Y`: Move Alice from X to Y (cost = edge weight)
- `move-e-X-Y`: Move Emil from X to Y (cost = edge weight)
- `move-f-X-Y`: Move Frank from X to Y (cost = edge weight)

5.2 DFS Execution

DFS on Meeting Point - Step by Step

Initial State: $(a = A, e = E, f = F)$
Action Ordering: Alphabetically by person, then by destination

Step 0: Initialize

Open (Stack)	State			g
	a	e	f	
n_0	A	E	F	0

Step 1: Expand n_0

Pop: $n_0 : (A, E, F)$
Goal Test: NO - different locations
Available Actions from Graph:

- Alice at A: can move to B (cost 1) or D (cost 1)
- Emil at E: can move to B (cost 2), D (cost 2), G (cost 3), H (cost 1)
- Frank at F: can move to D (cost 1), G (cost 4)

Generate Successors (alphabetically):

1. `move-a-A-B`: $(B, E, F), g = 1 \rightarrow n_1$
2. `move-a-A-D`: $(D, E, F), g = 1 \rightarrow n_2$
3. `move-e-E-B`: $(A, B, F), g = 2 \rightarrow n_3$
4. `move-e-E-D`: $(A, D, F), g = 2 \rightarrow n_4$
5. `move-e-E-G`: $(A, G, F), g = 3 \rightarrow n_5$
6. `move-e-E-H`: $(A, H, F), g = 1 \rightarrow n_6$
7. `move-f-F-D`: $(A, E, D), g = 1 \rightarrow n_7$
8. `move-f-F-G`: $(A, E, G), g = 4 \rightarrow n_8$

Open (Stack)	$[n_1, n_2, n_3, n_4, n_5, n_6, n_7, \mathbf{n_8}] \leftarrow$ top
Expanded	n_0

DFS: n_8 added last, so it's explored first!

Step 2: Expand n_8

Pop: $n_8 : (A, E, G)$ - Frank moved to G
Goal Test: NO
Available Actions:

- Alice at A: $\rightarrow$ B, D
- Emil at E: $\rightarrow$ B, D, G, H
- Frank at G: $\rightarrow$ E, F, H

Generate Some Successors:

- `move-a-A-B`: $(B, E, G), g = 5$
- `move-a-A-D`: $(D, E, G), g = 5$
- `move-e-E-G`: $(A, G, G), g = 7$ - Alice not there yet
- `move-f-G-E`: $(A, E, E), g = 7$ - Alice not there yet
- ... many more

DFS continues deep along Frank-to-G path before exploring other siblings

Step 3-7: DFS Deep Exploration

DFS continues exploring deeply along various branches. For example:
Step 3: Expand last added successor (e.g., move Frank from G to H)
Step 4: Continue deep on that branch
Step 5: Eventually hits dead-end or finds goal by chance
Step 6-7: Backtracks when necessary

Example Path DFS Might Explore:

1. $(A, E, F) \xrightarrow{f \rightarrow G} (A, E, G)$

2. $(A, E, G) \xrightarrow{f \to H} (A, E, H)$

3. $(A, E, H) \xrightarrow{e \to H} (A, H, H)$

4. $(A, H, H) \xrightarrow{a \to B} (B, H, H)$

5. ... continues until backtrack or lucky find

Key Point: DFS explores one complete path deeply before trying alternatives. Might take many steps or get lucky!

5.3 BFS Execution

BFS on Meeting Point - Step by Step

Initial State: $(a = A, e = E, f = F)$

Step 0: Initialize

Level	Node	a	e	f	g
0	n_0	A	E	F	0

Step 1: Expand n_0 - Level 0

Dequeue: $n_0 : (A, E, F)$
Goal Test: NO
Generate All Successors: (same 8 as DFS)

Node	a	e	f	g
n_1	B	E	F	1
n_2	D	E	F	1
n_3	A	B	F	2
n_4	A	D	F	2
n_5	A	G	F	3
n_6	A	H	F	1
n_7	A	E	D	1
n_8	A	E	G	4

Open (Queue)	$[\mathbf{n_1}, n_2, n_3, n_4, n_5, n_6, n_7, n_8]$
Level	1 - All depth-1 nodes

BFS: All level-1 nodes in queue. n_1 at front will be dequeued next.

Step 2: Expand n_1 - Level 1

Dequeue: $n_1 : (B, E, F)$ - Alice moved to B
Goal Test: NO
Available Actions:

- Alice at B: $\to$ A, C, D, E
- Emil at E: $\to$ B, D, G, H
- Frank at F: $\to$ D, G

Generate Successors (partial list):

- `move-a-B-E`: $(E, E, F), g = 3$ - Alice joins Emil!
- `move-e-E-B`: $(B, B, F), g = 3$ - Emil joins Alice!
- ... and others

These level-2 nodes are added to back of queue.

Step 3-8: Continue Level 1

BFS continues expanding all level-1 nodes: $n_2, n_3, n_4, n_5, n_6, n_7, n_8$
Each generates multiple level-2 nodes (two people at same location).
Important Level-2 nodes generated:

- States where 2 people meet: (D, D, F), (E, E, F), (A, E, D), etc.
- These are closer to goal!

Level 1 Complete! Queue now contains many level-2 nodes.

Step 9-10: Expand Level 2 - Two People Together

Example - Dequeue: (D, D, F) - Alice and Emil both at D
Goal Test: NO - Frank still at F
Available Actions:

- Move Alice from D
- Move Emil from D
- Move Frank from F: $\rightarrow$ D (cost 1), G (cost 4)

Key Successor:

- `move-f-F-D`: $(D, D, D), g = ?$ - **ALL AT D!**

This is a GOAL state! BFS will find it when expanding level-2 or level-3 nodes.
BFS Guarantee: Will find optimal meeting point with minimum total cost!

BFS Summary for Meeting Point

Level-by-Level Exploration:

- **Level 0:** Initial positions
- **Level 1:** One person moved
- **Level 2:** Two people moved (or one person moved twice)
- **Level 3:** Combinations bringing all three together

Optimal Solution Found: BFS finds the minimum-cost path to any (X, X, X) state.
Comparison:

- **DFS:** Explores one complete branch deeply, might find solution by luck or take many steps
- **BFS:** Systematically explores by distance, guaranteed to find optimal meeting point

6 Exercise 1.4: Elevator Scheduling Problem

6.1 Problem Formulation

Elevator Scheduling Problem

Problem: A single elevator serves a 3-floor building. Transport 3 passengers to their destinations with minimum travel distance.
Capacity: 2 passengers maximum (C = 2)
Passengers:

Passenger	Source	Destination
1	1	3
2	1	2
3	3	2

State Representation: (e, p_1, p_2, p_3) where:

- $e \in \{1, 2, 3\}$ = elevator floor
- $p_i \in \{1, 2, 3, e\}$ = passenger i location (e means in elevator)

Initial State: $(e = 1, p_1 = 1, p_2 = 1, p_3 = 3)$
Goal States: $(e = ?, p_1 = 3, p_2 = 2, p_3 = 2)$ - all at destinations
Actions:

- `up-X-Y`: Move elevator up (cost = 1 per floor)
- `down-X-Y`: Move elevator down (cost = 1 per floor)
- `board-pX-fY`: Passenger X boards at floor Y (cost = 0)
- `exit-pX-fY`: Passenger X exits at floor Y (cost = 0)

6.2 DFS Execution

DFS on Elevator - Step by Step

Initial State: $(e = 1, p_1 = 1, p_2 = 1, p_3 = 3)$
Action Ordering: board actions, then exit actions, then up, then down

Step 0: Initialize

Node	e	p_1	p_2	p_3	g
n_0	1	1	1	3	0

Step 1: Expand n_0

Pop: $n_0 : (1, 1, 1, 3)$
Elevator at floor 1, passengers 1 and 2 waiting at floor 1, passenger 3 at floor 3
Goal Test: NO - passengers not at destinations
Available Actions:

- `board-p1-f1`: Passenger 1 boards (capacity allows)
- `board-p2-f1`: Passenger 2 boards (capacity allows)
- `up-1-2`: Elevator goes up to floor 2

Generate Successors:

1. `board-p1-f1`: $(1, e, 1, 3), g = 0 \rightarrow n_1$
2. `board-p2-f1`: $(1, 1, e, 3), g = 0 \rightarrow n_2$
3. `up-1-2`: $(2, 1, 1, 3), g = 1 \rightarrow n_3$

Open (Stack)	$[n_1, n_2, \mathbf{n_3}] \leftarrow$ top
Expanded	n_0

DFS: n_3 (moving elevator without boarding) explored first!

Step 2: Expand n_3

Pop: $n_3 : (2, 1, 1, 3)$ - Elevator at floor 2, no passengers aboard
Goal Test: NO
Available Actions:

- `up-2-3`: Move to floor 3 (cost 1)
- `down-2-1`: Move back to floor 1 (cost 1)

Generate Successors:

1. `down-2-1`: $(1, 1, 1, 3), g = 2$ - back to initial!
2. `up-2-3`: $(3, 1, 1, 3), g = 2 \rightarrow n_4$

Open (Stack)	$[n_1, n_2, \mathbf{n_4}] \leftarrow$ top
Expanded	n_0, n_3

Inefficient: DFS wasting moves by going up without passengers!

Step 3: Expand n_4

Pop: $n_4 : (3, 1, 1, 3)$ - Elevator at floor 3, passenger 3 waiting there
Goal Test: NO
Available Actions:

- `board-p3-f3`: Passenger 3 boards
- `down-3-2`: Move to floor 2

Generate Successors:

1. `board-p3-f3`: $(3, 1, 1, e), g = 2 \rightarrow n_5$
2. `down-3-2`: $(2, 1, 1, 3), g = 3 \rightarrow n_6$

Stack: $[n_1, n_2, n_5, \mathbf{n_6}]$

Step 4: Expand n_6

Pop: $n_6 : (2, 1, 1, 3)$ - Elevator at floor 2
Goal Test: NO
Continue generating successors...

DFS continues exploring, sometimes making inefficient moves.

Step 5-7: DFS Continues

Eventually DFS might explore:
Step 5: Go back and try $n_5 : (3, 1, 1, e)$ - passenger 3 in elevator at floor 3
Step 6: From n_5, move down: $(2, 1, 1, e), g = 3$
Step 7: Exit passenger 3: $(2, 1, 1, 2), g = 3$ - Passenger 3 delivered!

Progress: After many steps, DFS might deliver one passenger, but path may not be optimal.
DFS Characteristic: Explores deeply, may make many unnecessary elevator moves before finding good solution.

6.3 BFS Execution

BFS on Elevator - Step by Step

Initial State: $(e = 1, p_1 = 1, p_2 = 1, p_3 = 3)$

Step 0: Initialize

Level	Node	e	p_1	p_2	p_3	g
0	n_0	1	1	1	3	0

Step 1: Expand n_0 - Level 0

Dequeue: $n_0 : (1, 1, 1, 3)$
Goal Test: NO
Generate Successors:

- $n_1 : (1, e, 1, 3), g = 0$ - board p1
- $n_2 : (1, 1, e, 3), g = 0$ - board p2
- $n_3 : (2, 1, 1, 3), g = 1$ - move up

Open (Queue)	$[\mathbf{n_1}, n_2, n_3]$
Level	1

BFS: Board actions have cost 0, so same level as initial. Will explore these first!

Step 2: Expand n_1 - Level 0 (g

Dequeue: $n_1 : (1, e, 1, 3)$ - Passenger 1 in elevator
Goal Test: NO
Available Actions:

- `board-p2-f1`: Board passenger 2 (capacity = 2 allows)
- `exit-p1-f1`: Exit passenger 1 (but destination is floor 3!)
- `up-1-2`: Move elevator up

Generate Successors:

1. `board-p2-f1`: $(1, e, e, 3), g = 0 \rightarrow n_4$ - Both p1, p2 aboard!

2. `up-1-2`: $(2, e, 1, 3), g = 1 \rightarrow n_5$

Queue: $[n_2, n_3, \mathbf{n_4}, n_5]$

Step 3: Expand n_2 - Level 0

Dequeue: $n_2 : (1, 1, e, 3)$ - Passenger 2 in elevator
Similar to n_1, generates:

- $(1, e, e, 3), g = 0$ - same as n_4 (both aboard)
- $(2, 1, e, 3), g = 1$ - elevator up with p2

Step 4: Expand n_4 - Level 0 (g

Dequeue: $n_4 : (1, e, e, 3)$ - Both p1 and p2 in elevator
Goal Test: NO
Capacity: Full (2/2)
Available Actions:

- `up-1-2`: Move to floor 2 (cost 1)
- Cannot board p3 - capacity full

Generate Successors:

- `up-1-2`: $(2, e, e, 3), g = 1 \rightarrow n_7$

Good Progress: Both passengers from floor 1 now aboard!

Step 5: Expand n_3 - Level 1 (g

Dequeue: $n_3 : (2, 1, 1, 3)$ - Elevator at floor 2, passengers left behind
This is inefficient path (moved without passengers)
BFS continues, but this branch is not optimal.

Step 6: Expand n_7 - Level 1 (g

Dequeue: $n_7 : (2, e, e, 3)$ - Elevator at floor 2 with p1 and p2
Goal Test: NO
Available Actions:

- `exit-p2-f2`: Drop passenger 2 at destination floor 2!
- `up-2-3`: Continue to floor 3
- `down-2-1`: Back to floor 1

Generate Successors:

1. `exit-p2-f2`: $(2, e, 2, 3), g = 1 \rightarrow n_8$ - P2 delivered!
2. `up-2-3`: $(3, e, e, 3), g = 2 \rightarrow n_9$

Excellent: Can deliver p2 at cost 1!

Step 7: Expand n_8 - Level 1 (g

Dequeue: $n_8 : (2, e, 2, 3)$ - Elevator at floor 2, p1 aboard, p2 delivered!
Goal Test: NO - still need to deliver p1 and p3
Available Actions:

- `up-2-3`: Move to floor 3 (cost 1)
- `down-2-1`: Move to floor 1 (cost 1)

Generate Successors:

- `up-2-3`: $(3, e, 2, 3), g = 2 \rightarrow n_{10}$ - At floor 3!

Step 8-10: Continue to Goal

Step 8: Expand n_{10}: $(3, e, 2, 3), g = 2$ - Elevator at floor 3 with p1
Actions:

- `exit-p1-f3`: $(3, 3, 2, 3), g = 2$ - P1 delivered!
- `board-p3-f3`: $(3, e, 2, e), g = 2$ - But p1 still aboard!

Step 9: After exiting p1: $(3, 3, 2, 3), g = 2$

- `board-p3-f3`: $(3, 3, 2, e), g = 2 \rightarrow n_{11}$

Step 10: From n_{11}: $(3, 3, 2, e), g = 2$ - P3 aboard

- `down-3-2`: $(2, 3, 2, e), g = 3 \rightarrow n_{12}$

Step 11: From n_{12}: $(2, 3, 2, e), g = 3$

- `exit-p3-f2`: $(2, 3, 2, 2), g = 3$ - **GOAL!**

Total Cost: 3 (only elevator moves count)

BFS Summary for Elevator

Optimal Solution Found:

1. Board p1 and p2 at floor 1 (cost 0)
2. Move up to floor 2 (cost 1)
3. Exit p2 at floor 2 (cost 0)
4. Move up to floor 3 (cost 1)
5. Exit p1 at floor 3 (cost 0)
6. Board p3 at floor 3 (cost 0)
7. Move down to floor 2 (cost 1)
8. Exit p3 at floor 2 (cost 0)

Total Travel Distance: 3 floors
BFS Advantage:

- Systematically explores by cost (g-value)

- Board actions (cost 0) explored first
- Finds optimal schedule minimizing elevator travel

DFS Disadvantage:

- Might make unnecessary elevator moves
- No guarantee of optimality
- Could waste time exploring inefficient paths

7 Final Comparison: DFS vs BFS

Property	DFS	BFS
Data Structure	Stack (LIFO)	Queue (FIFO)
Exploration Order	Depth-first	Level-by-level
Optimality	No	Yes (unit cost)
Completeness	No (infinite paths)	Yes
Space Complexity	$O(b \cdot d)$	$O(b^d)$
Time Complexity	$O(b^d)$	$O(b^d)$
Memory Efficient	Yes	No
Finds Shortest Path	No	Yes

7.1 Key Insights from Exercises

Exercise 1.2 - Team Assignment

- **DFS:** Dives deep into one team configuration, may miss better ones nearby
- **BFS:** Systematically tries all combinations at each level, finds balanced team at shallowest depth
- **Winner:** BFS for systematic exploration

Exercise 1.3 - Meeting Point

- **DFS:** Might send all three people on long journeys before finding meeting point
- **BFS:** Explores by total distance, guaranteed to find minimum-cost meeting point
- **Winner:** BFS for optimal solution

Exercise 1.4 - Elevator Scheduling

- **DFS:** May make inefficient elevator moves (e.g., going up empty)
- **BFS:** Explores by travel cost, finds minimum distance schedule
- **Winner:** BFS for cost-optimal scheduling

7.2 When to Use Which?

Algorithm Selection Guide

Use DFS when:

- Memory is severely limited
- Solutions are deep in the tree
- Any solution is acceptable (don't need optimal)
- Search space is infinite and you need to go deep

Use BFS when:

- Need optimal (shortest) solution
- Solutions are relatively shallow
- Enough memory available
- Want to explore systematically by levels
- Cost optimization is important

8 Conclusion

This document demonstrated detailed step-by-step execution of DFS and BFS on three different problem types:

1. **Team Assignment:** Combinatorial optimization
2. **Meeting Point:** Multi-agent path finding
3. **Elevator Scheduling:** Resource-constrained scheduling

Key Takeaway: Understanding the data structure (Stack vs Queue) is the key to understanding the algorithm's behavior. DFS's stack makes it go deep, while BFS's queue makes it explore breadth-first, level-by-level.